

ARTIFICIAL INTELLIGENCE

Lecture Set 07
Constraint Satisfaction Problems

Dr. Maria Siddiqua
Assistant Professor
Department of AI & DS
FAST-NUCES Karachi
maria.siddiqua@nu.edu.pk

Course Contents

1. Introduction to AI
2. Agents and Environments
3. Problem Solving by Searching
4. Uninformed Search
5. Informed Search
6. Local Search
- **7. Constraint Satisfaction Problems**
8. Adversarial Search
9. Logical Agents
10. First-order Logic
11. Uncertain Knowledge and Reasoning
12. Supervised, Unsupervised, and Reinforcement Learning
13. Recent Trends in AI and Applications of AI Algorithms

Table of Contents

- Constraint Satisfaction Problem (CSP)
- Constraint Graph
- CSP Examples
- Backtrack Search
- Improving Backtrack Efficiency
- Arc Consistency
- Tree-structured CSP

Constraint Satisfaction Problem (CSP)

Standard Search Problem >>

- ▶ Searches a space of states for the solution.
- ▶ Heuristics are domain specific.
- ▶ Each state is atomic/indivisible i.e. a black box with no internal structure.

CSP >>

- ▶ CSP uses a factored representation for each state: a set of variables, each of which has a value.
- ▶ A problem is solved when each variable has a value that satisfies all the constraints in the variable.
- ▶ A problem described this way is called a *constraint satisfaction problem*.

Advantages

- ▶ Uses general-purpose rather than problem-specific heuristics to enable the solution of complex problems.
- ▶ Represents each state as a set of variables and takes advantage of the structure of states.
- ▶ Eliminates large portions of the search space all at once by identifying variable/value combinations that violate the constraints.

CSP >> Formal Definition

A constraint satisfaction problem consists of three components, X , D , and C :

- ▶ X is the set of variables, $\{X_1, \dots, X_n\}$
- ▶ D is a set of domains, $\{D_1, \dots, D_n\}$, one for each variable.
- ▶ C is a set of constraints that specify allowable combinations of values.

Here,

- ▶ Each domain D_i consists of a set of allowable values, $\{v_1, \dots, v_k\}$ for each variable X_i
- ▶ Each constraint C_i consists of a pair $\langle \text{scope}, \text{rel} \rangle$, where *scope* is a tuple of variables that participate in the constraint and *rel* is a relation that defines the values that those variables can take on.
- ▶ A relation can be represented as an *explicit* list of all tuples of values that satisfy the constraint, or as an *abstract* relation. For example,

$\langle (X_1, X_2), [(A, B), (B, A)] \rangle$ or $\langle (X_1, X_2), X_1 \neq X_2 \rangle$

CSP >> Solution

- ▶ Each state in a CSP is defined by an assignment of values to some or all of the variables.

$$\{ X_i = v_i, X_j = v_j, \dots \}$$

- ▶ An assignment that does not violate any constraints is called a **consistent or legal assignment**.
- ▶ A **complete assignment** is one in which every variable is assigned.
- ▶ A **partial assignment** is one that assigns values to only some of the variables.
- ▶ A **solution** to a CSP is consistent and complete assignment.

Advantages of formulating a problem as a CSP

- ▶ CSPs yield natural representation for a wide variety of problems.
- ▶ CSP solvers can be faster than state-space searchers because the CSP solver can quickly eliminate large swatches of the search space.
- ▶ With CSP, once we find out that a partial assignment is not a solution, we can immediately discard further refinements of the partial assignment.
- ▶ We can see why a assignment is not a solution – which variables violate a constraint – we can focus attention on the variables that matter.

CSP >> Types of Variables

Discrete variables >>

- ▶ Finite domains:
 - ▶ n variables, each domain size d ; so $O(d^n)$ complete assignments.
 - ▶ e.g. Map coloring problem, 8-queens problem.
- ▶ Infinite domains:
 - ▶ Set of integers, strings, etc.
 - ▶ e.g. job scheduling problem with no deadline, variables are start/end days for each job.
 - ▶ Need a constraint language to describe constraints, ex. $\text{StartJob1} + 5 \leq \text{StartJob3}$

Continuous variables >>

- ▶ e.g., start/end times for Hubble Telescope observations
- ▶ linear constraints solvable in poly time by LP methods

CSP >> Types of Constraints

- ▶ **Unary constraints** involve a single variable,
e.g., $SA \neq \text{green}$
- ▶ **Binary constraints** involve pairs of variables,
e.g., $SA \neq WA$
only binary constraints can be represented as a constraint graph.
- ▶ **Higher-order constraints** involve 3 or more variables,
e.g., $SA \neq WA \neq NT$
- ▶ **Global constraints** involve arbitrary number of variables,
e.g. *alldiff* that represents all of the variables involved in the constraint must have different values.

CSP >> Problem Formulation

Map Coloring Problem >>

your task is to color each region either *red*, *green*, or *blue* in such a way that no neighboring regions have the same color.

CSP Formulation

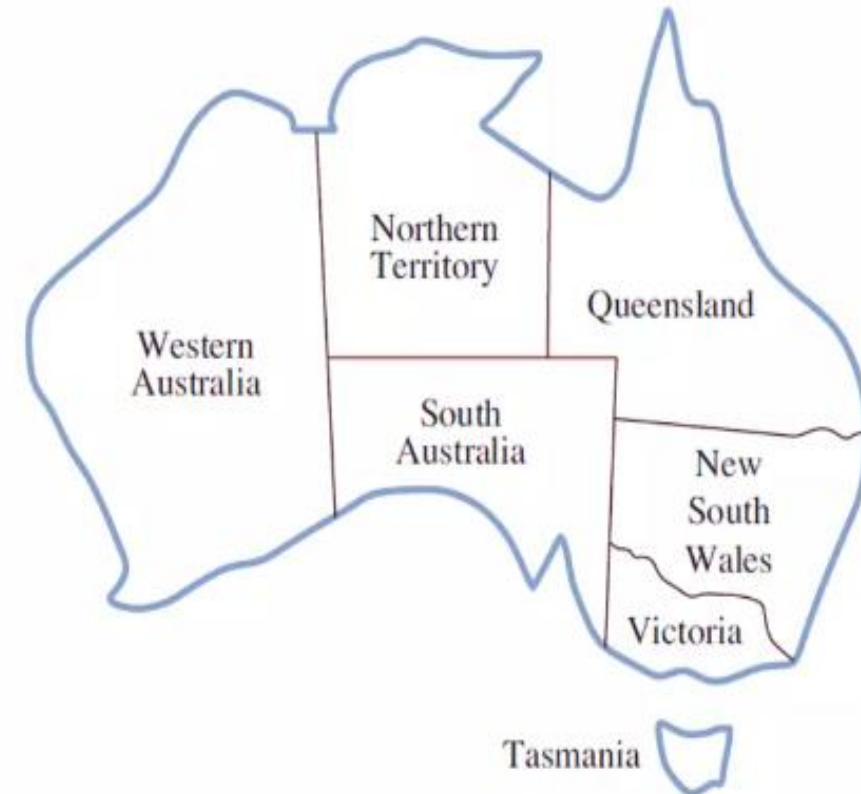
Each region is a variable, $X = \{ WA, NT, Q, NSW, V, SA, T \}$

Domain of each variable, $D_i = \{ \text{red, green, blue} \}$

Set of constraints,

$C = \{ SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, \\ WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V \}$

Here, $SA \neq WA$ is a shortcut for $\langle \text{scope}, \text{rel} \rangle = \langle (SA, WA), SA \neq WA \rangle$

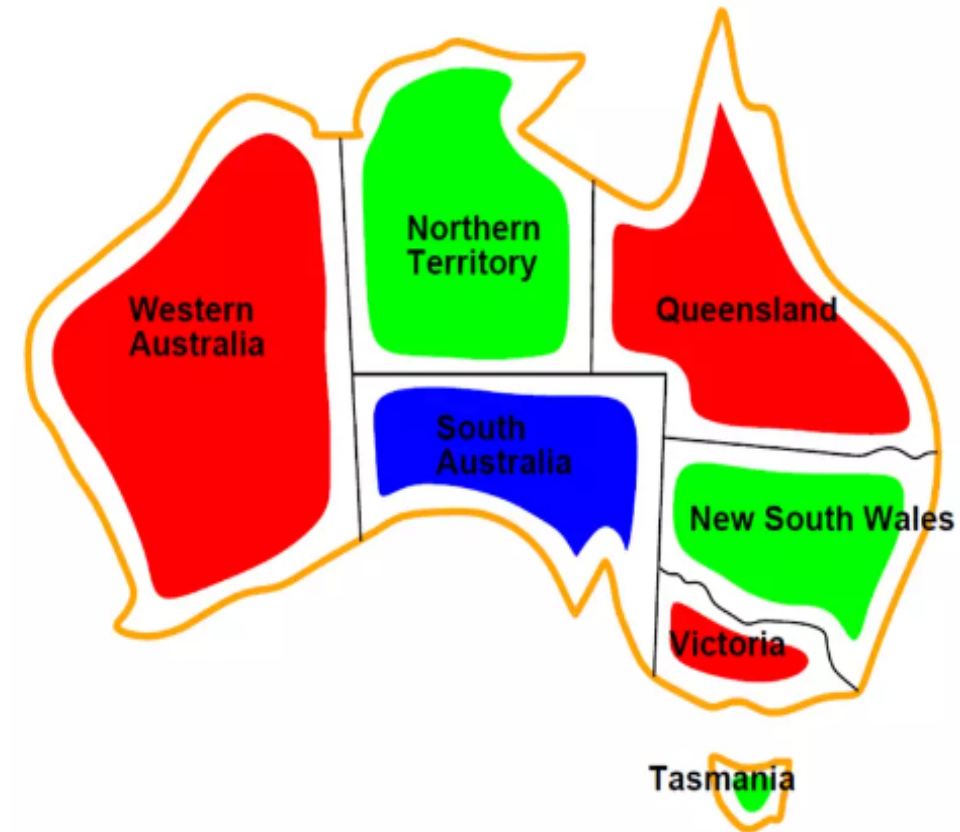


CSP >> Possible Solution

There are many possible solutions to this problem,

Such as

{ WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green }

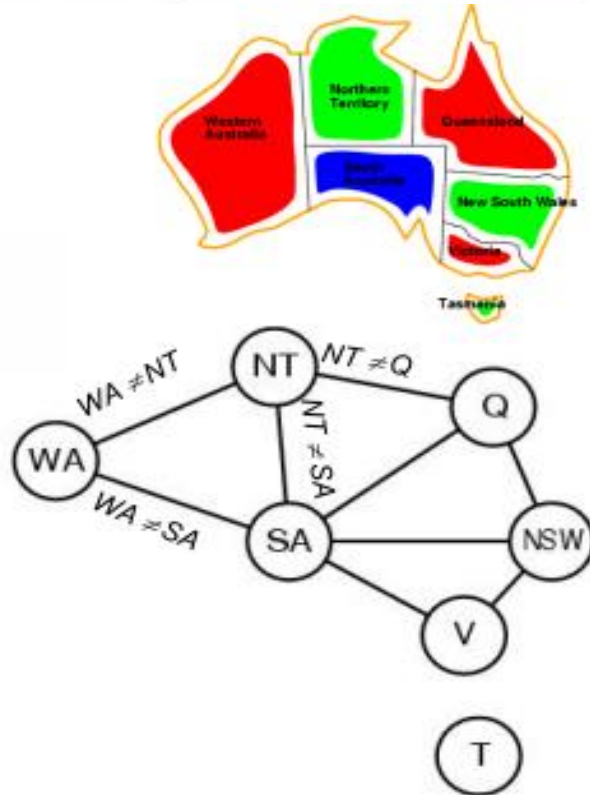


CSP >> Constraint Graph

Binary CSP >> each constraint relates at most two variables.

Constraint Graph >>

- ▶ Nodes correspond to the variables of the problem.
- ▶ An edge connects any two variables that participate in a constraint.



CSP >> Example (Sudoku)

Variables, X = each open square

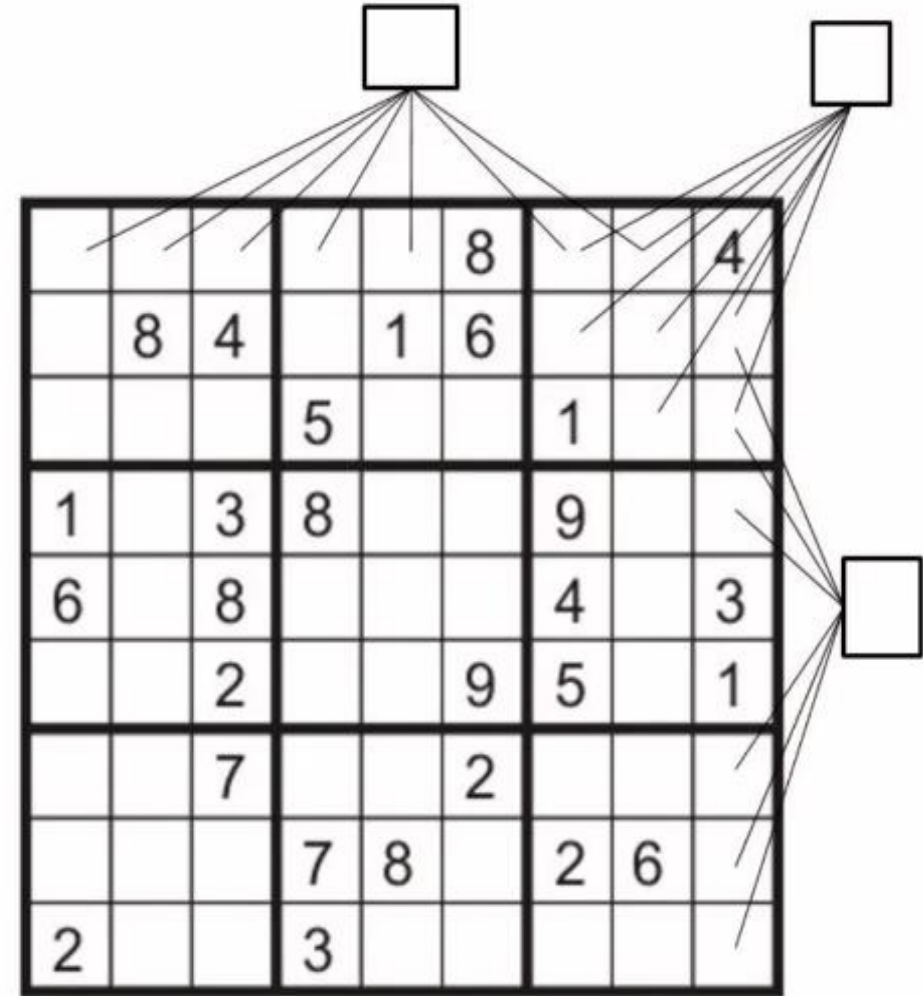
Domains for each variable, $D_i = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Constraints:

9-way alldiff for each column

9-way alldiff for each row

9-way alldiff for each region



CSP >> Example (Time-table Scheduling)

You are in charge of scheduling for computer science classes that meet Mondays, Wednesdays and Fridays. There are 5 classes that meet on these days and 3 professors who will be teaching these classes. You are constrained by the fact that each professor can only teach one class at a time.

The classes are:

- ▶ Class 1 - Intro to Programming: meets from 8:00-9:00am
- ▶ Class 2 - Intro to Artificial Intelligence: meets from 8:30-9:30am
- ▶ Class 3 - Natural Language Processing: meets from 9:00-10:00am
- ▶ Class 4 - Computer Vision: meets from 9:00-10:00am
- ▶ Class 5 - Machine Learning: meets from 9:30-10:30am

The professors are:

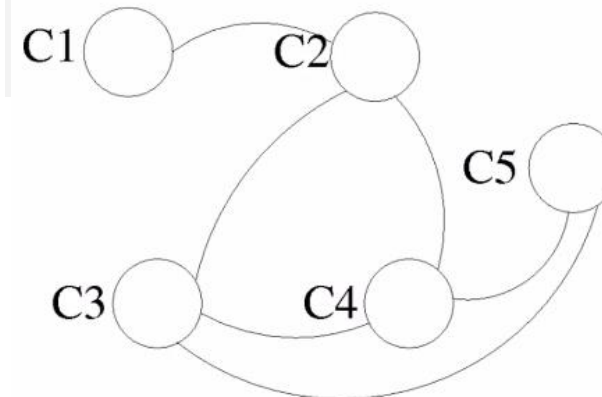
- ▶ Professor A, who is available to teach Classes 3 and 4.
- ▶ Professor B, who is available to teach Classes 2, 3, 4, and 5.
- ▶ Professor C, who is available to teach Classes 1, 2, 3, 4, 5

Variables	Domains
C1	C
C2	B, C
C3	A, B, C
C4	A, B, C
C5	B, C

Constraints:

$C1 \neq C2$
 $C2 \neq C3$
 $C3 \neq C4$
 $C4 \neq C5$
 $C2 \neq C4$
 $C3 \neq C5$

Constraint Graph



Solution

$C1 = C$

$C2 = B$

$C3 = A$

$C4 = C$

$C5 = B$

CSP >> Homework (4 House Problem-Who Lives Where)

There are four families A, B, C, D living in four houses 1, 2, 3, and 4.

Given the following constraints, find out which family lives in which house.

Constraints:

1. C lives in a house with higher number than D.
 2. D lives next to A in a lower number house.
 3. There is at least one house between D and B.
 4. C does not live in 3.
 5. B does not live in 1.
-
- a) Formulate this problem as a CSP problem in which there is one variable per family, stating the domains, and constraints (write unary and binary constraints).
 - b) Draw the constraint graph associated with this CSP.
 - c) Show the domains of the variables after running arc-consistency on this initial graph (after having already enforced any unary constraints).
 - d) Give one solution to this CSP.

CSP >> Example (Cryptarithmic Puzzle)

Variables, $X = \{ F, T, U, W, R, O, X_1, X_2, X_3 \}$

Domains for variables X_1, X_2, X_3 is, $\{ 0, 1 \}$

Domains for all other variables, $D_i = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 \}$

Constraints:

$alldiff(F, T, U, W, R, O)$

$O + O = R + 10 \cdot X_1$

$X_1 + W + W = U + 10 \cdot X_2$

$X_2 + T + T = O + 10 \cdot X_3$

$X_3 = F, T \neq 0, F \neq 0$

$$\begin{array}{r} T \ W \ O \\ + \ T \ W \ O \\ \hline F \ O \ U \ R \end{array}$$

Homework:

In each of these puzzles, the challenge is to find the correct digit-to-letter mapping that satisfies the arithmetic equation, no two letters can be assigned same digit.

$$\begin{array}{r} B \ A \ S \ E \\ + B \ A \ L \ L \\ \hline G \ A \ M \ E \ S \end{array}$$

$$\begin{array}{r} P \ I \ N \ K \\ + B \ L \ U \ E \\ \hline W \ H \ I \ T \ E \end{array}$$

$$\begin{array}{r} C \ R \ A \ C \ K \\ + H \ A \ C \ K \\ \hline E \ R \ R \ O \ R \end{array}$$

$$\begin{array}{r} S \ E \ N \ D \\ + M \ O \ R \ E \\ \hline M \ O \ N \ E \ Y \end{array}$$

$$\begin{array}{r} W \ A \ T \ E \ R \\ + J \ U \ I \ C \ E \\ \hline D \ R \ I \ N \ K \end{array}$$

CSP >> Solving CSPs

We can formulate CSP problems as standard search problems.

- ▶ **States:** defined by the values assigned so far
- ▶ **Initial state:** the empty assignment { }
- ▶ **Successor function:** assign value to variable without conflict
-fail, if no legal assignments possible
- ▶ **Goal test:** the current assignment is complete
- ▶ **Path cost:** constant step cost

Standard search algorithms can be applied directly to solve CSPs.

Let, n variables and the domain size for each variable is d

For BFS:

Branching factor, $b = nd$

Nodes at level 1 = nd

Nodes at level 2 = $nd * (n-1) d$

Nodes/leaves at level $n = nd * (n-1)d * \dots * d = n! * d^n$

CSP >> Backtracking Search

A crucial property to all CSPs: **commutativity**

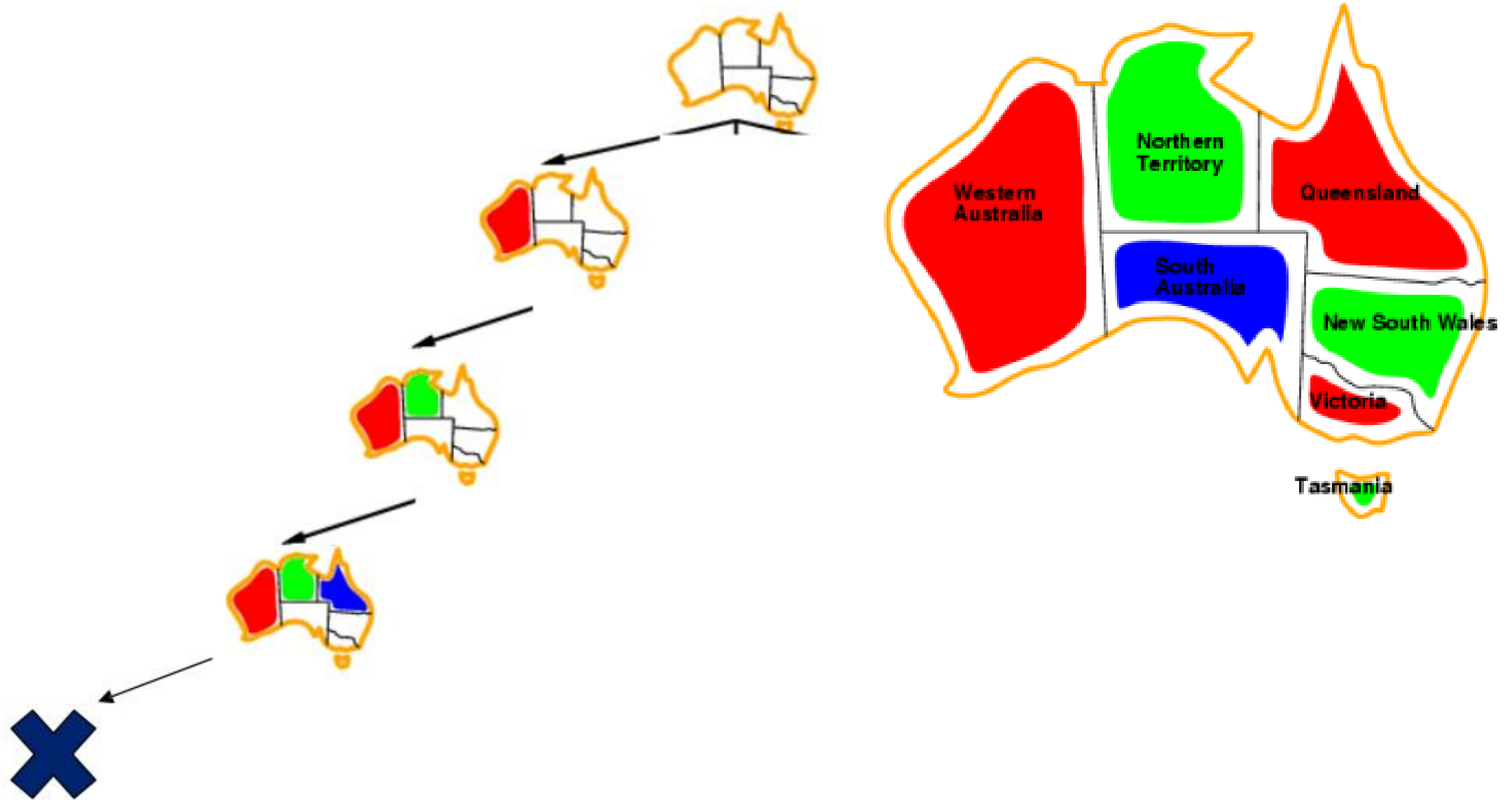
- ▶ The order of application of any given set of actions has no effect on the outcome.
- ▶ Variable assignments are commutative, i.e. [WA=red then NT=green] same as [NT=green then WA=red]

So now need to consider only assignments to a single variable at each node. So branching factor becomes $b=d$ and total no of leaves reduces to b^d

Depth-first search for CSPs with single-variable assignment and backtracks when a variable has no legal values left to assign is called **Backtracking search**.

- ▶ Backtracking search is used for a depth-first search that chooses values for one variable at a time and backtracks when a variable has no legal values left to assign.
- ▶ Backtracking search is the basic uninformed algorithm for CSPs.

CSP >> Backtracking Search Example



CSP >> Improving Backtracking Efficiency

General-purpose methods can give huge gains in speed:

Issue 1 >> Which variable should be assigned next?

sol >> Minimum remaining values heuristic

sol >> Degree heuristic

Issue 2 >> In what order should its values be tried?

sol >> Least constraining value heuristic

Issue 3 >> Can we detect inevitable failure early?

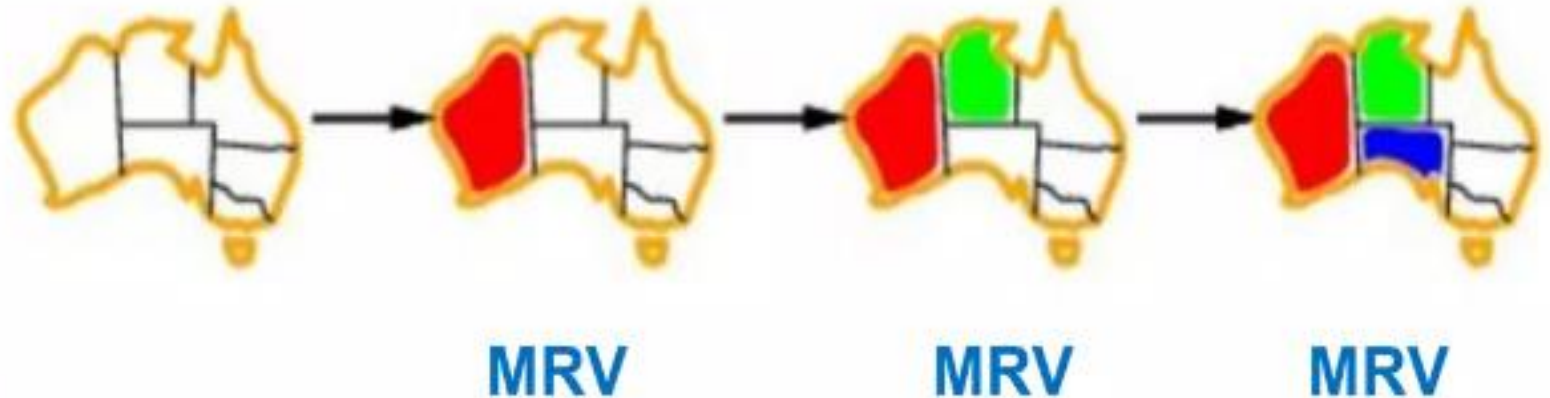
sol >> Forward checking constraint propagation

sol >> Maintaining Arc-Consistency inference

CSP >> Variable Selection [MRV Heuristic]

Minimum remaining values / Most constrained variable / Fail-first heuristic >>

- ▶ Choose the variable with the fewest legal values.



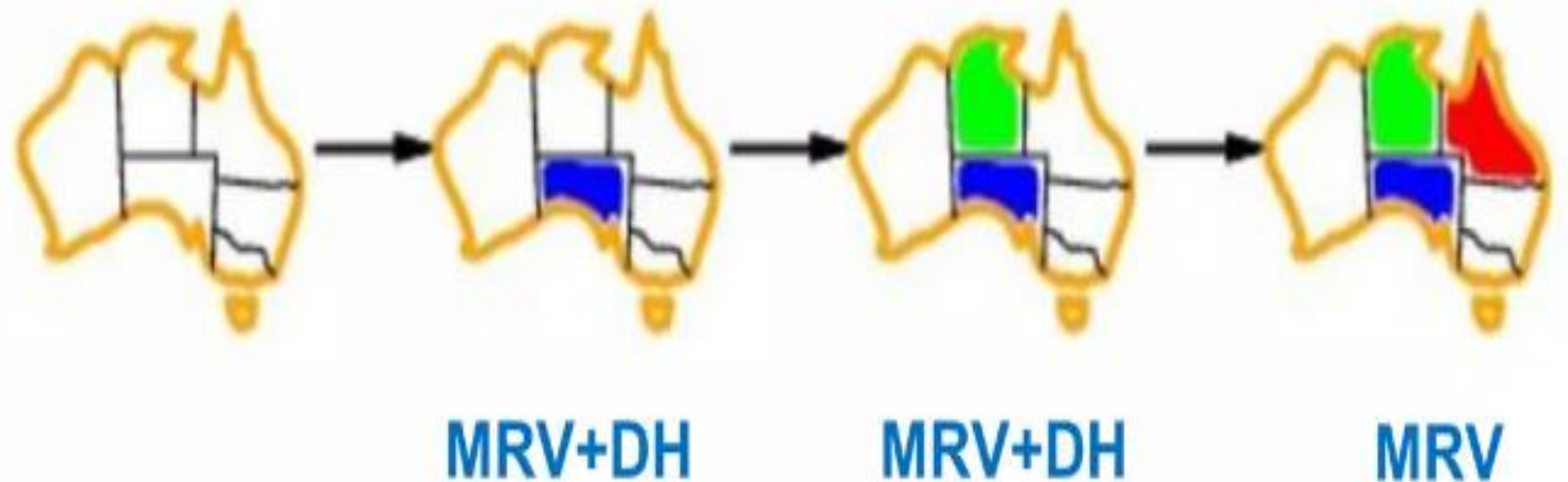
Can't answer the following questions:

- ▶ Which variable to choose first?
- ▶ Which variable to choose if they have the same minimum remaining number of legal values?

CSP >> Variable Selection [Degree Heuristic]

Degree / Most constraining variable heuristic >>

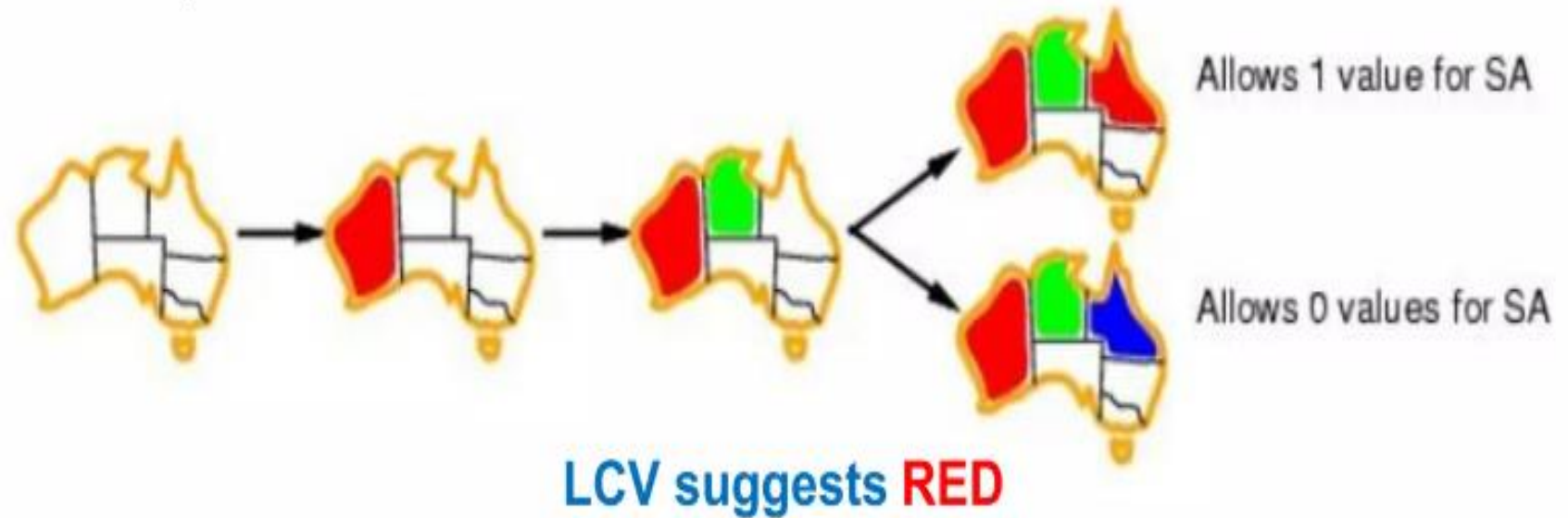
- ▶ Selecting the variable that has the largest number of constraints on other unassigned variables.
- ▶ Tie-breaker among MRV



CSP >> Value Ordering [Least Constraining Value]

Least-constraining-value heuristic >>

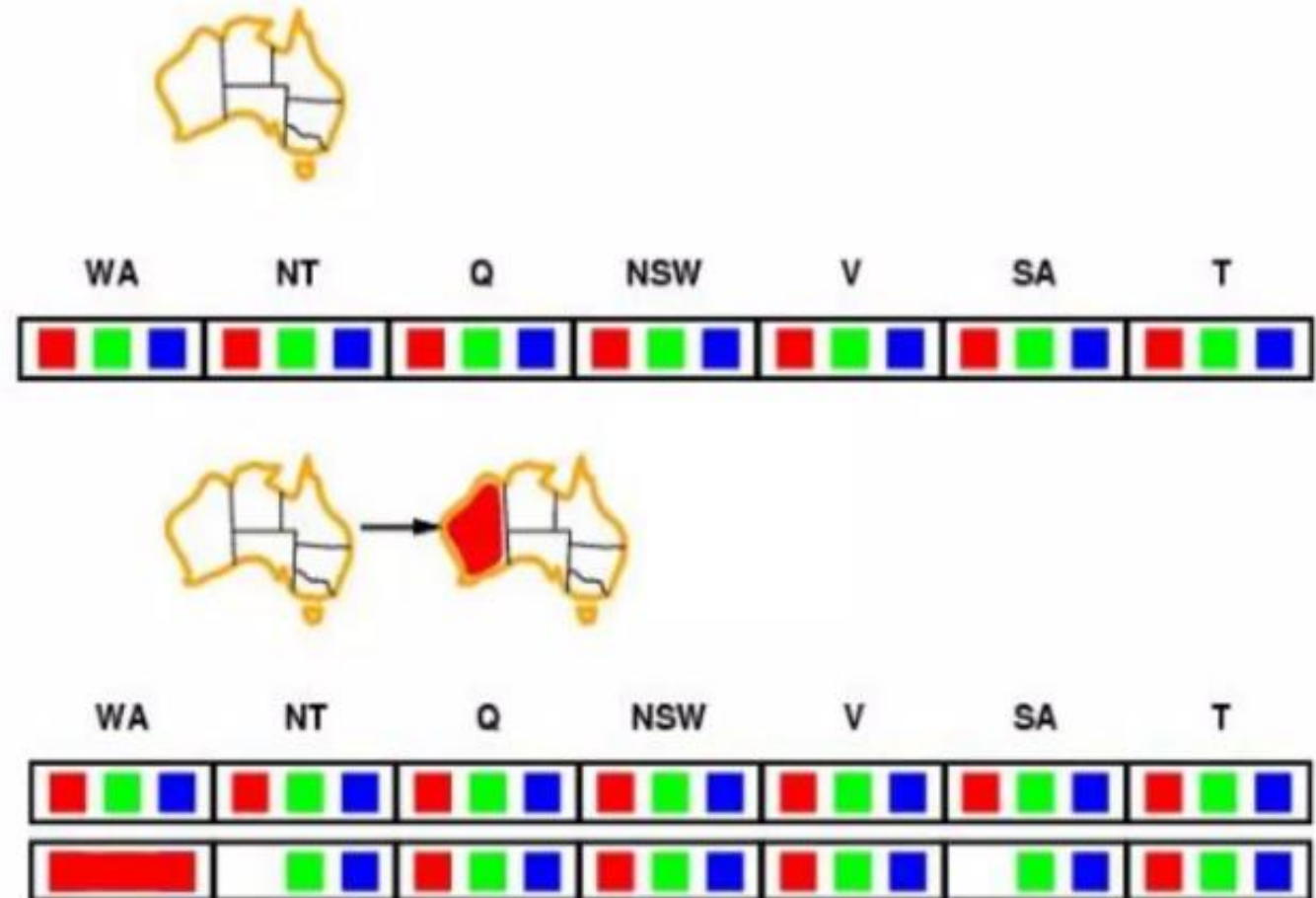
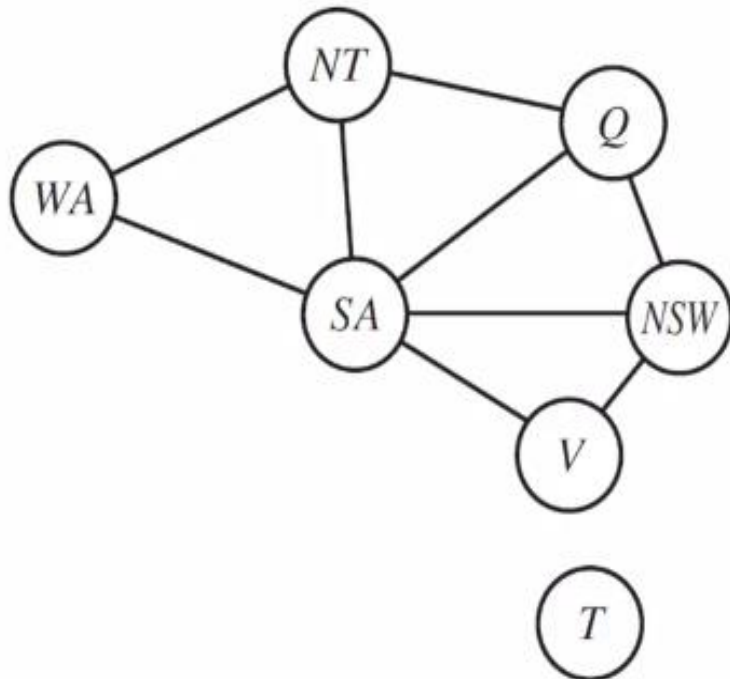
- ▶ Choose the value that rules out fewest choices for the neighboring variables in the constraint graph.
- ▶ Leaves maximum flexibility for the neighbors.



CSP >> Early Failure Detection [Forward Checking]

Forward-checking Inference >>

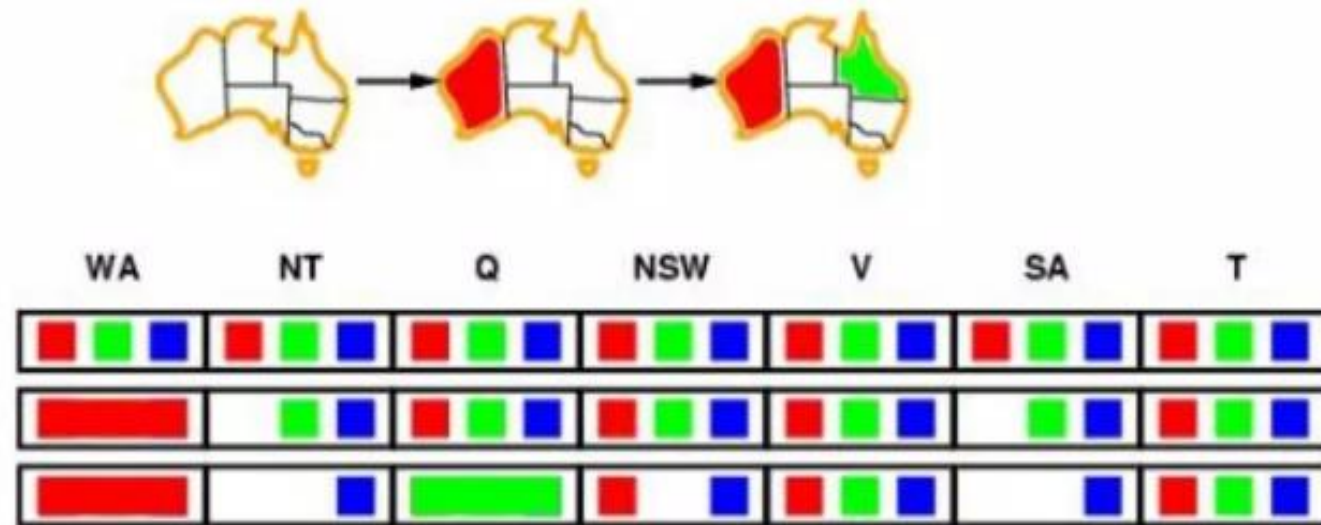
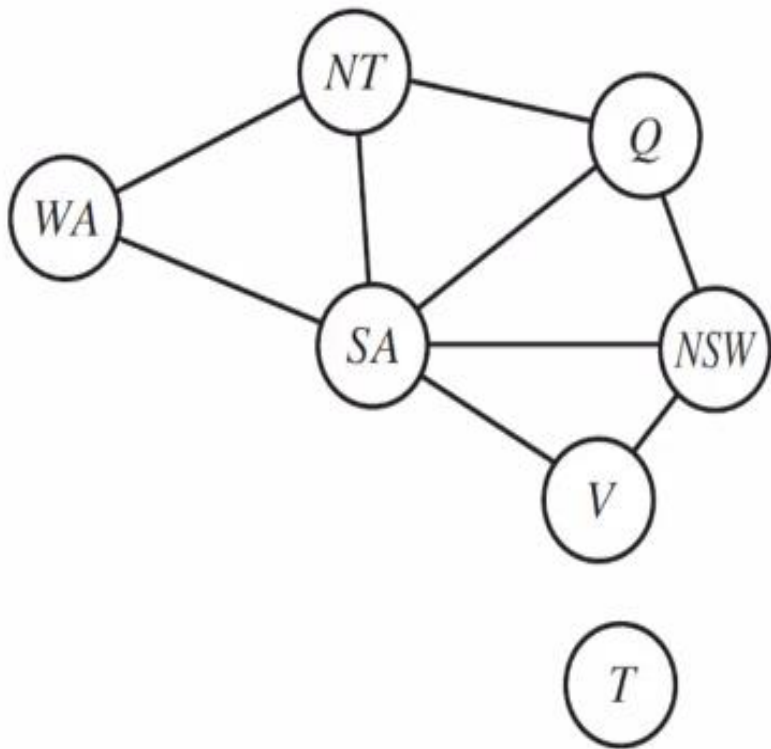
- ▶ Keep track of remaining legal values for unassigned variables.
- ▶ Terminate search when any variable has no legal values.



CSP >> Early Failure Detection [Forward Checking]

Forward-checking Inference >>

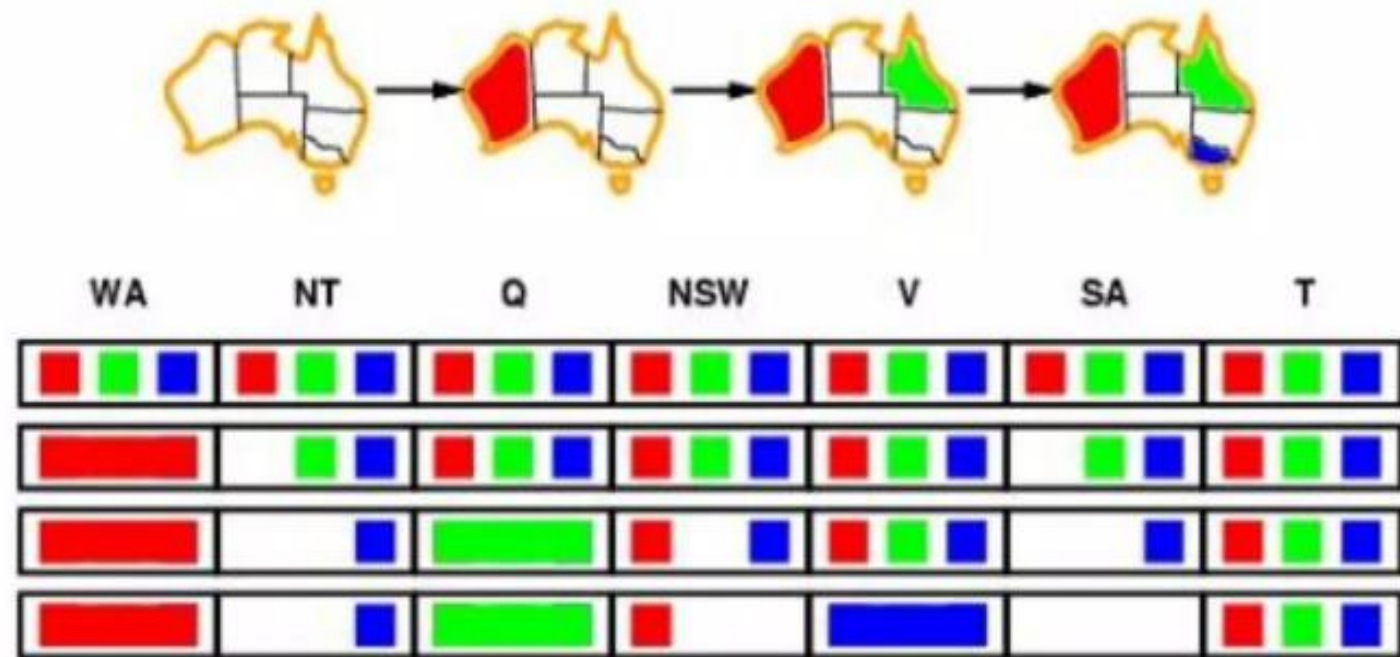
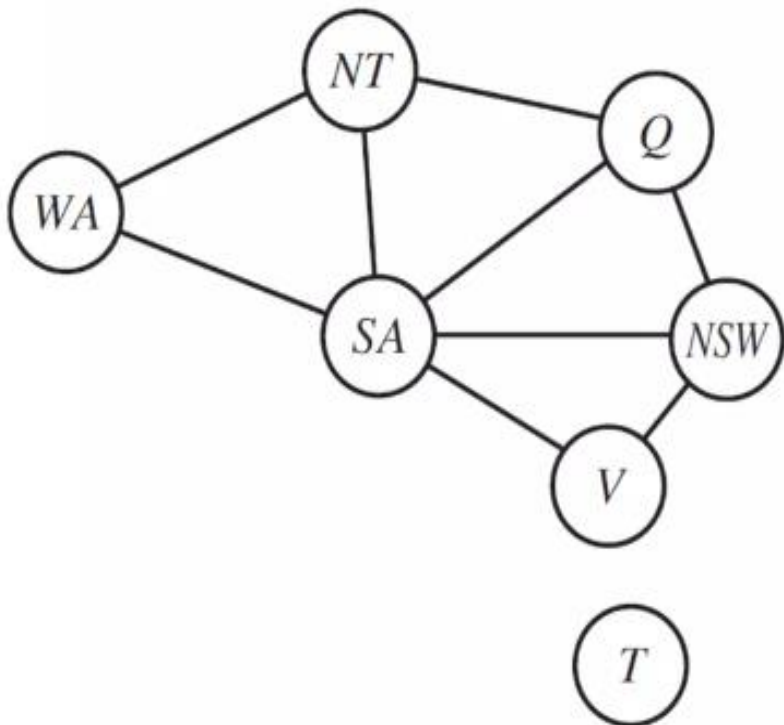
- ▶ Keep track of remaining legal values for unassigned variables.
- ▶ Terminate search when any variable has no legal values.



CSP >> Early Failure Detection [Forward Checking]

Forward-checking Inference >>

- ▶ Keep track of remaining legal values for unassigned variables.
- ▶ Terminate search when any variable has no legal values.



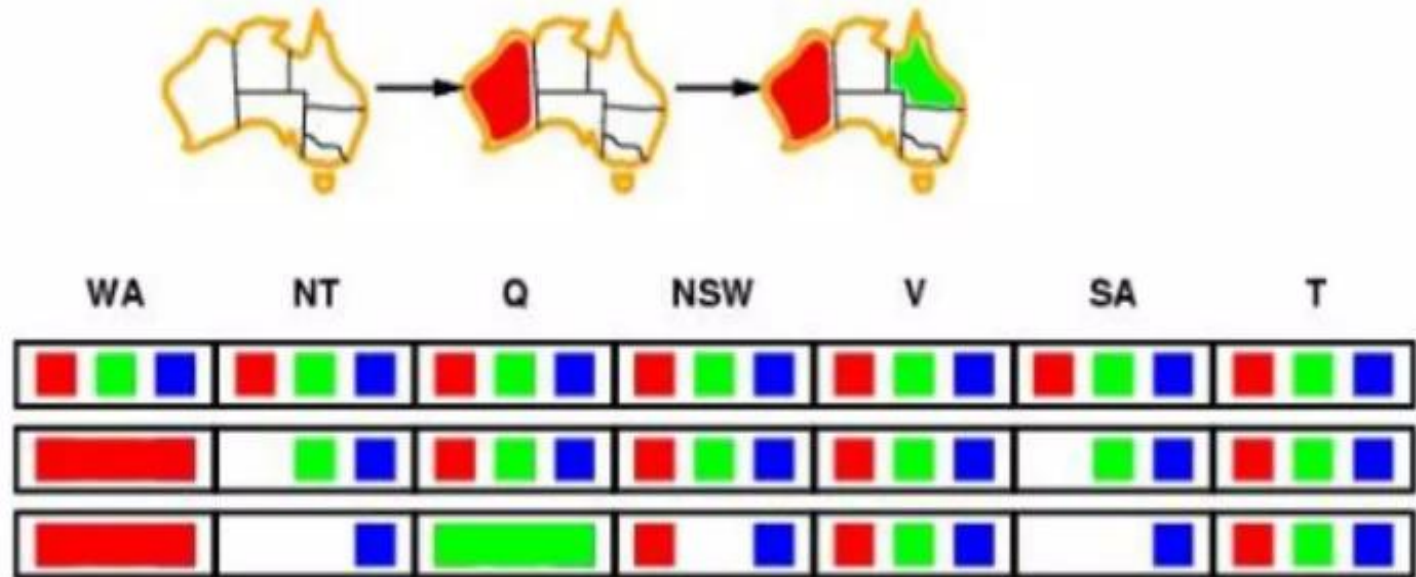
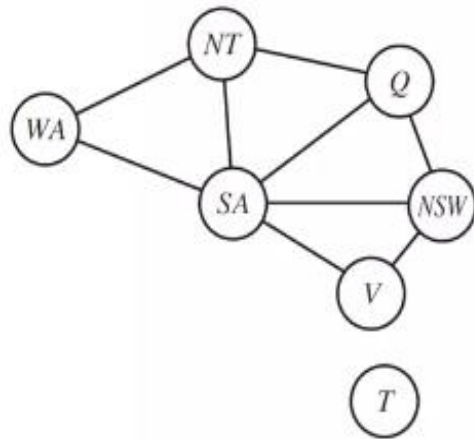
CSP >> Early Failure Detection [Forward Checking]

Forward-checking Inference >>

- ▶ Keep track of remaining legal values for unassigned variables.
- ▶ Terminate search when any variable has no legal values.

Problems >>

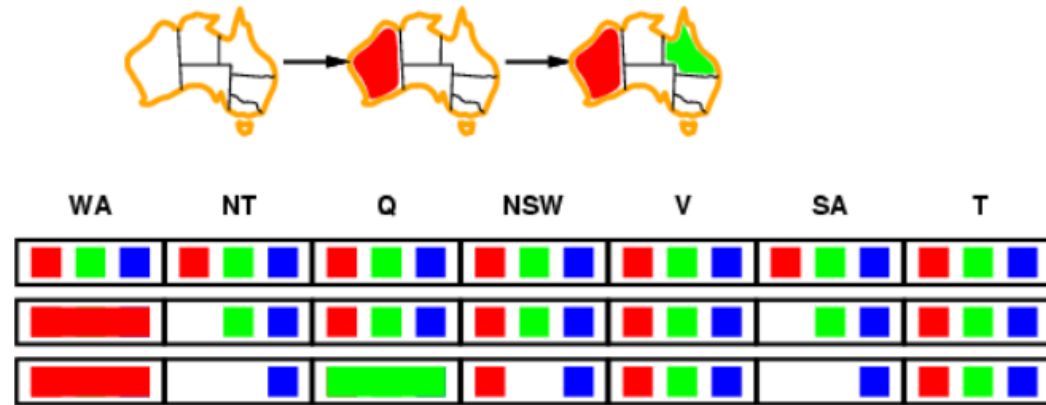
- ▶ FC only propagates information from assigned to unassigned variables, but doesn't provide early detection for all failures.
- ▶ For example, NT and SA can't both be blue!



Towards Constraint propagation



- **Forward checking** propagates information from *assigned to unassigned* variables, but doesn't provide early detection for all failures:



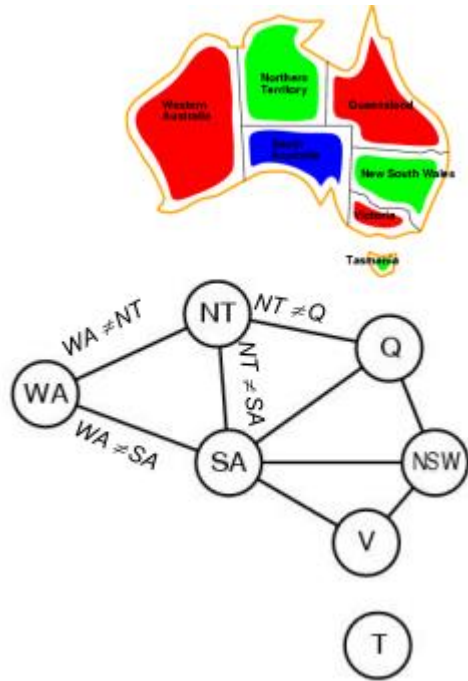
- NT and SA cannot both be blue!
- **Constraint propagation** goes beyond forward checking & repeatedly enforces constraints locally

CSP >> Constraint Propagation (Local Consistency)

1. **Node consistency:** If values in the variable's domain satisfy the variable's unary constraints.
2. **Arc consistency:** If every value in variable's domain satisfies the variable's binary constraints.
3. **Path consistency**
4. **K consistency**
5. **Global constraints (Sudoku)**

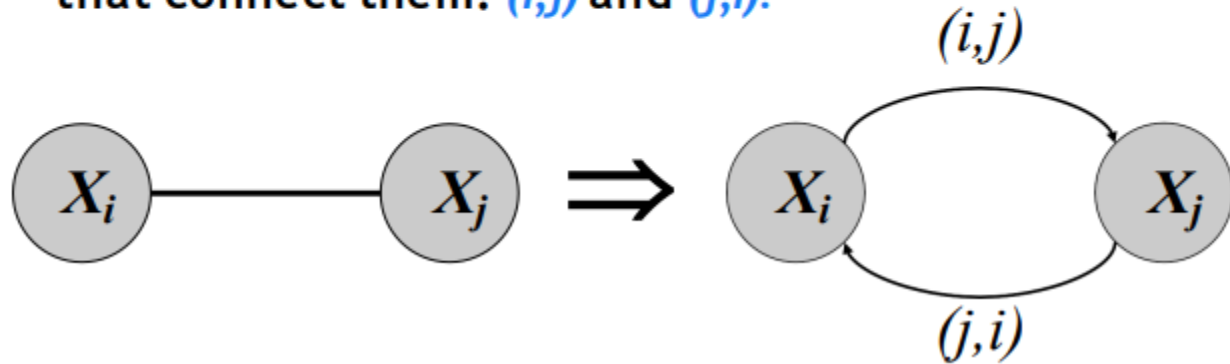
Edges to Arcs: From Constraint Graph to Directed Graph

- **Constraint graph:**
nodes are variables
edges are constraints



- Given a pair of nodes X_i and X_j connected by a constraint **edge**, we represent this not by a single undirected edge, but a **pair of directed arcs**.

For a connected pair of nodes X_i and X_j , there are **two** arcs that connect them: (i,j) and (j,i) .

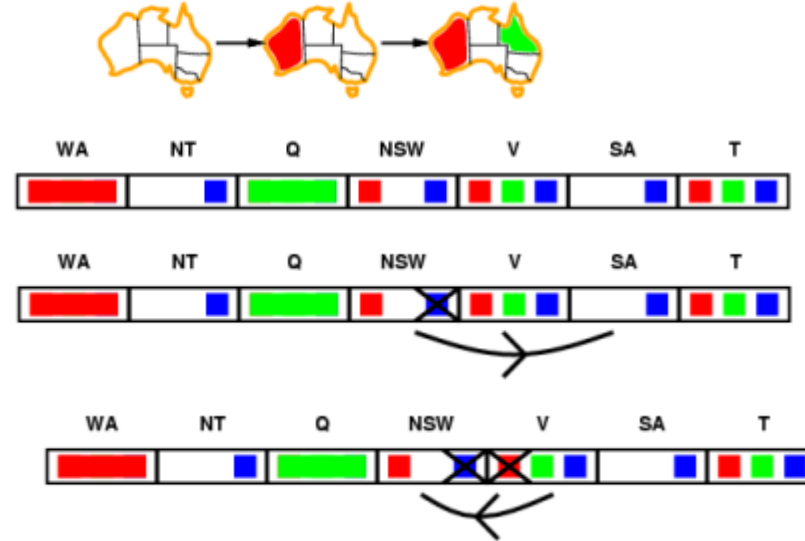


Arc consistency: satisfies binary constraints

- (X_i is arc-consistent w.r.t. X_j if for every value v in D_i , there is some value w in D_j that satisfies the binary constraint on the arc between X_i and X_j)

Arc consistency

- Simplest form of propagation makes each arc **consistent**
- $X \rightarrow Y$ is consistent iff
for **every** value x of X there is **some** allowed y

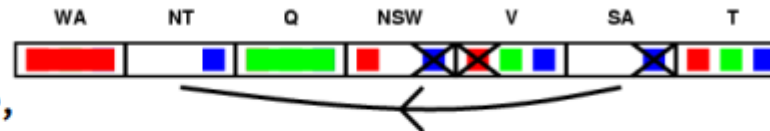


An arc (i,j) is **arc consistent** if and only if every value v on X_i is consistent with some label on Y_j .

To make an arc (i,j) arc consistent,
for each value v on X_i ,
if there is no label on Y_j consistent with v
then remove v from X_i

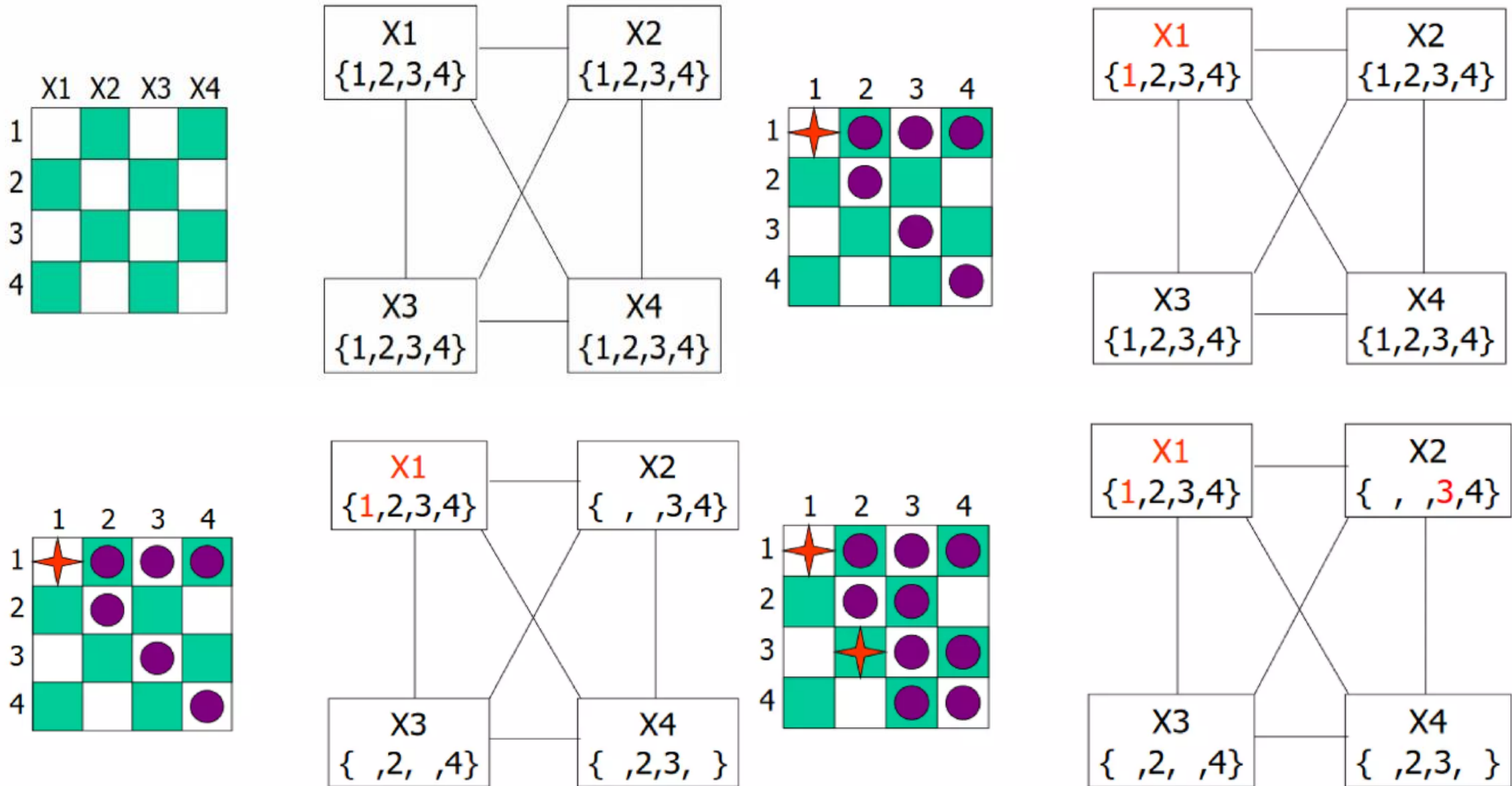
- Given d values, checking arc (i,j) takes $O(d^2)$ time worst case

- If X loses a value, recheck neighbors of X

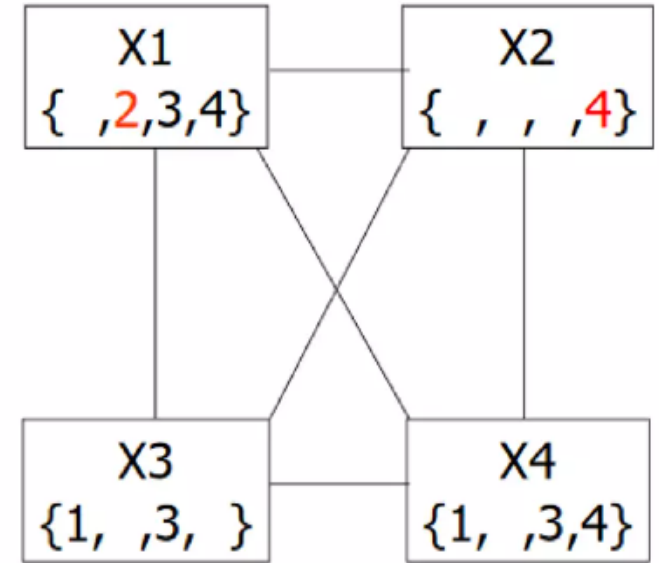
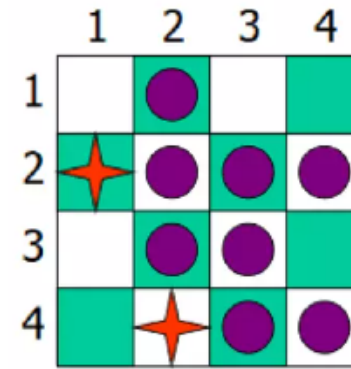
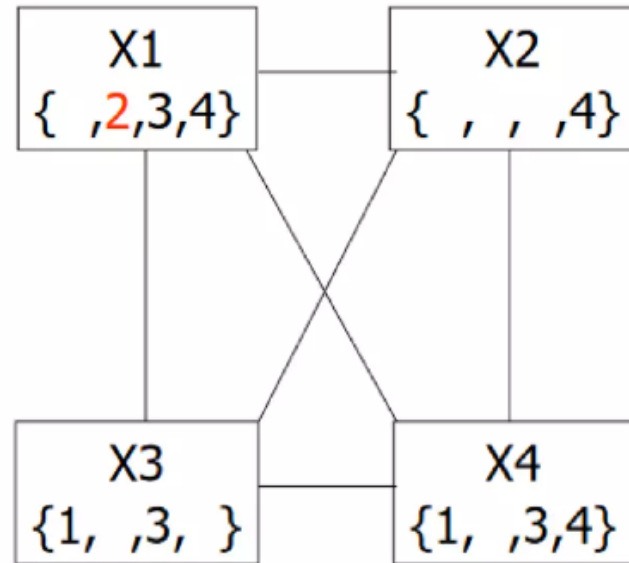
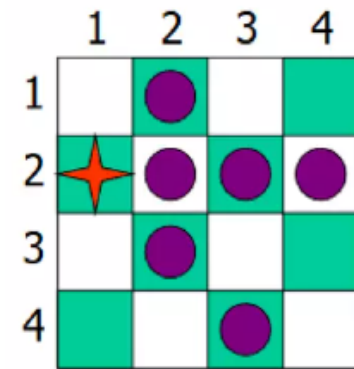
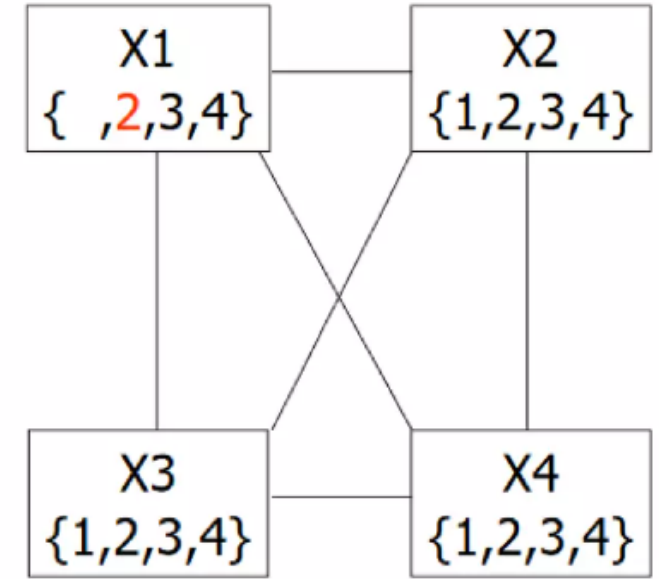
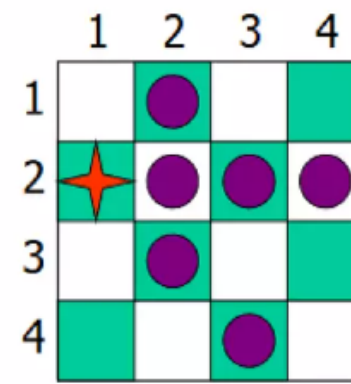
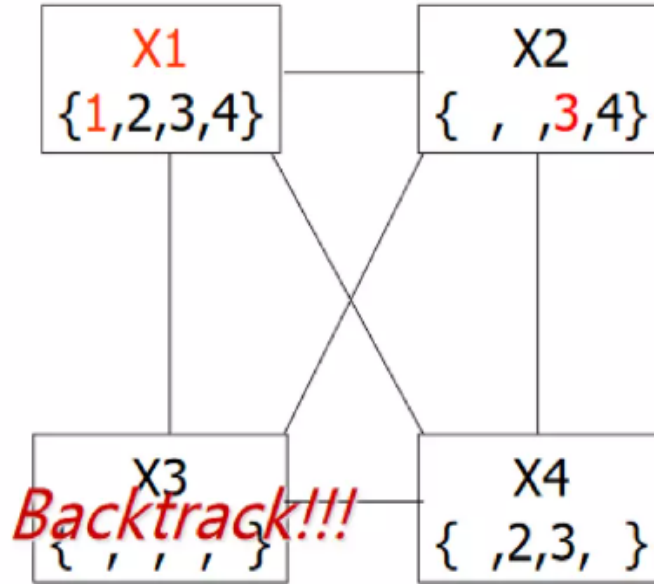
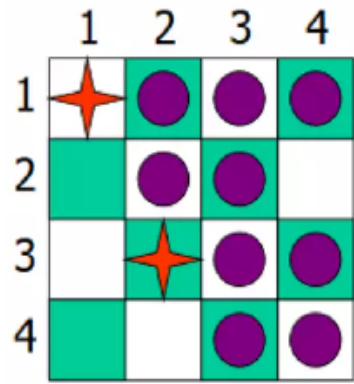


- If X loses a value,
- Detects failure earlier than forward checking
- Can be run as a preprocessor or after each assignment

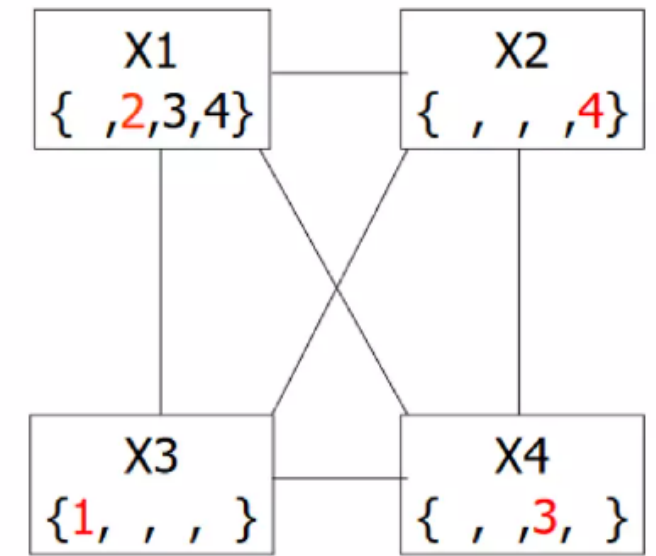
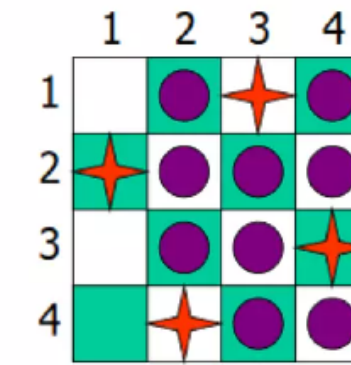
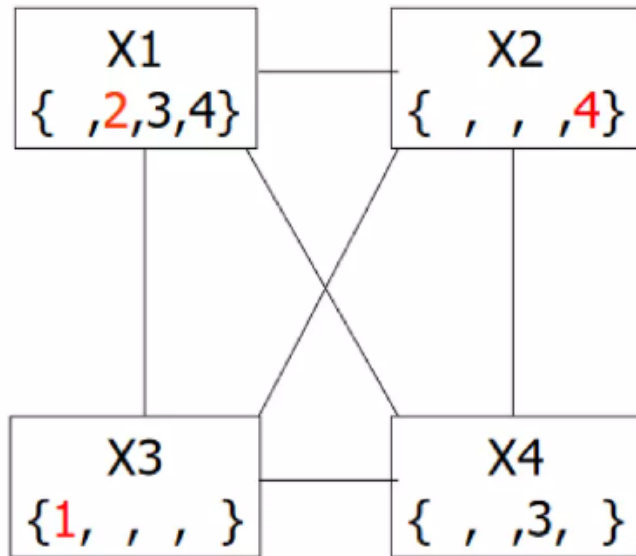
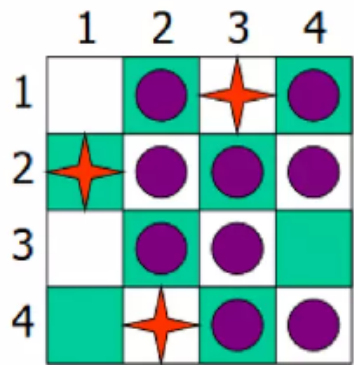
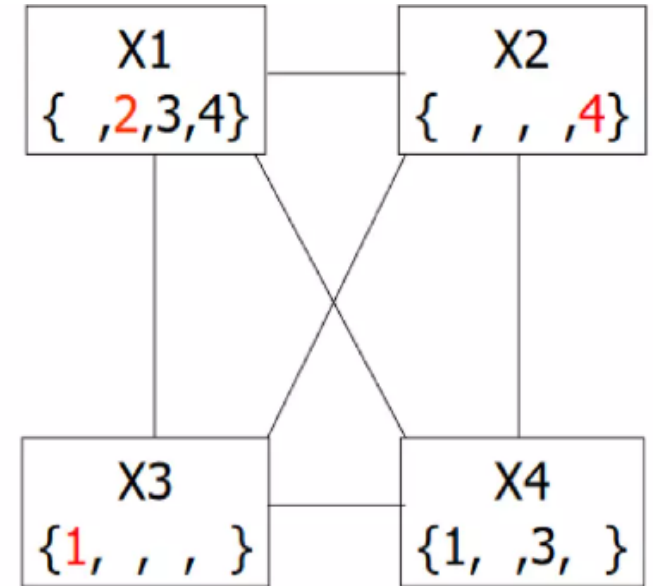
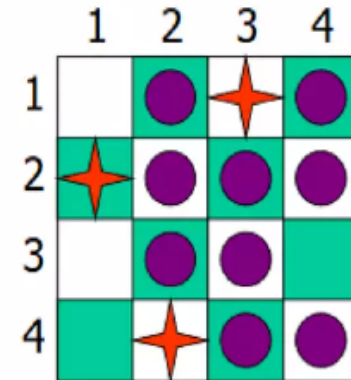
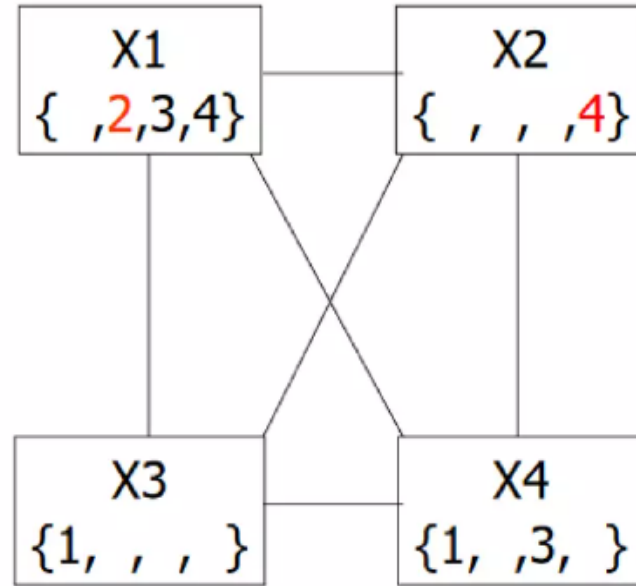
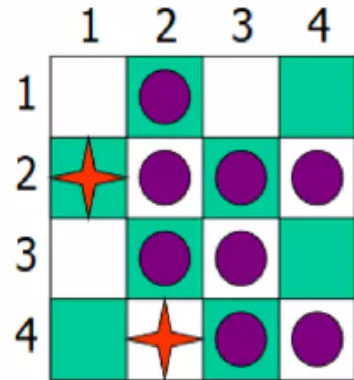
CSP >> Example [Forward Checking + Backtracking]



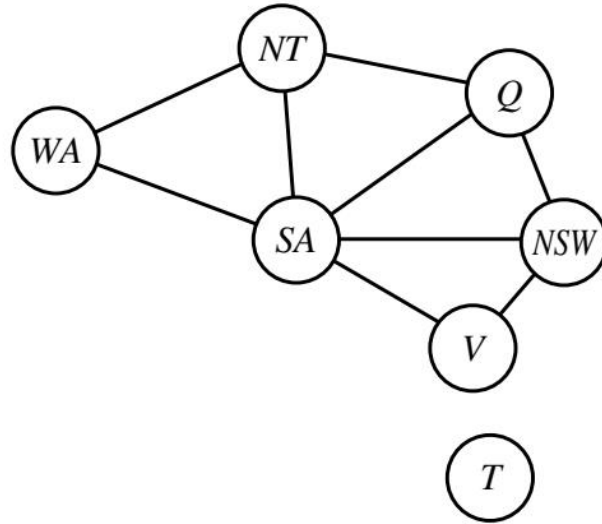
CSP >> Example [Forward Checking + Backtracking]



CSP >> Example [Forward Checking + Backtracking]



Problem structure

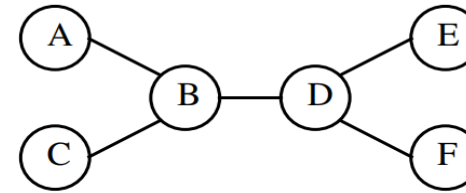


Tasmania and mainland are *independent subproblems*
Identifiable as *connected components* of constraint graph

Suppose each subproblem has c variables out of n total
Worst-case solution cost is $n/c \cdot d^c$, **linear** in n
E.g., $n = 80$, $d = 2$, $c = 20$

$2^{80} = 4$ billion years at 10 million nodes/sec
 $4 \times 2^{20} = 0.4$ seconds at 10 million nodes/sec

Tree-structured CSPs



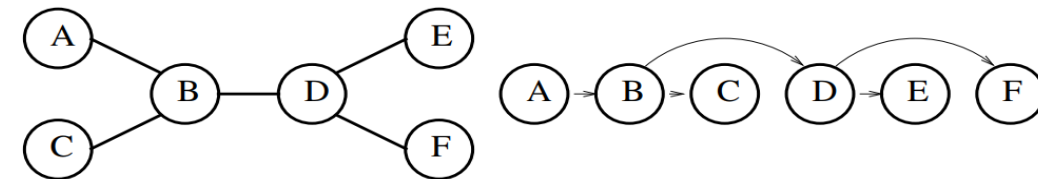
Theorem: if the constraint graph has no loops, the CSP can be solved in $O(nd^2)$ time

Compare to general CSPs, where worst-case time is $O(d^n)$

This property also applies to logical and probabilistic reasoning:
an important example of the relation between syntactic restrictions
and the complexity of reasoning.

Algorithm for tree-structured CSPs

1. Choose a variable as root, order variables from root to leaves such that every node's parent precedes it in the ordering



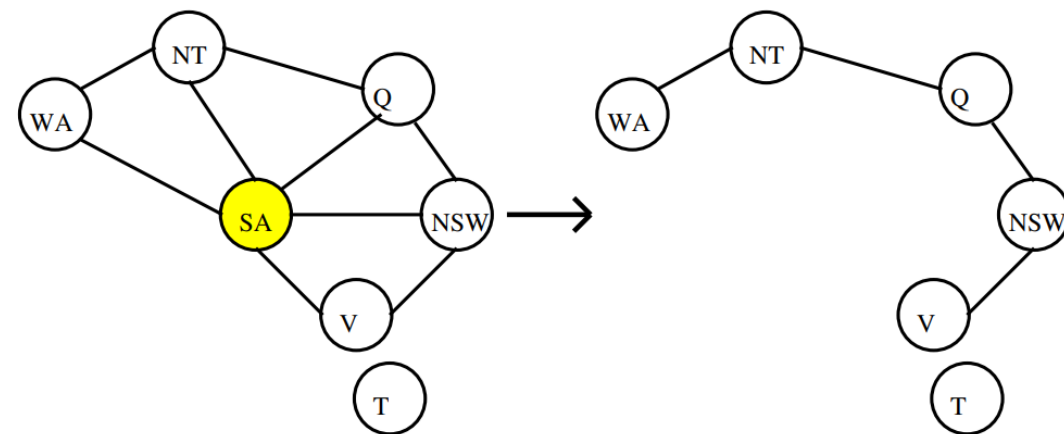
2. For j from n down to 2, apply
MAKE-ARC-CONSISTENT($Parent(X_j), X_j$)
(will remove inconsistent values)
3. For i from 1 to n , assign X_i consistently with $Parent(X_i)$

Algorithm for tree-structured CSPs (cont'd)

```
function TREE-CSP-SOLVER (csp)  
returns a solution, or failure  
inputs: csp, a binary CSP with components  $(X, D, C)$   
  
   $n \leftarrow$  number of variables in  $X$   
  assignment  $\leftarrow$  an empty assignment  
  root  $\leftarrow$  any variable in  $X$   
   $X \leftarrow \text{TOPOLOGICALSORT}(X, \text{root})$   
  for  $j = n$  down to 2 do  
    MAKE-ARC-CONSISTENT(Parent( $X_j$ ),  $X_j$ )  
    if it cannot be made consistent then return failure  
    for  $i = 1$  to  $n$  do  
      assignment [ $X_i$ ]  $\leftarrow$  any consistent value from  $D_i$   
      if there is no consistent value then return failure  
  return assignment
```

Nearly tree-structured CSPs

Conditioning: instantiate a variable, prune its neighbors' domains



Cutset conditioning: instantiate (in all ways) a set of variables such that the remaining constraint graph is a tree

Cutset size $c \implies$ runtime $O(d^c \cdot (n - c)d^2)$, very fast for small c

CSP >> Summary

- ▶ CSPs are a special kind of problem: states defined by values of a fixed set of variables goal test defined by constraints on variable values
- ▶ Backtracking = depth-first search with one variable assigned per node
- ▶ Variable ordering and value selection heuristics help significantly
- ▶ Forward checking prevents assignments that guarantee later failure
- ▶ Constraint propagation (e.g., arc consistency) does additional work to constrain values and detect inconsistencies
- ▶ The CSP representation allows analysis of problem structure
- ▶ Tree-structured CSPs can be solved in linear time

CSP >> Few Real World Examples

1. **Nurse Scheduling:** Assigning shifts to nurses while satisfying constraints such as skill levels, preferences, and legal working hour limits.
2. **Course Timetabling:** Scheduling courses, classrooms, and instructors to optimize resource utilization and minimize conflicts.
3. **Vehicle Routing:** Planning optimal routes for delivery vehicles to minimize fuel costs and delivery times while considering capacity constraints and customer time windows.
4. **Job Shop Scheduling:** Scheduling production jobs on machines to minimize makespan and maximize resource utilization while adhering to precedence constraints.
5. **University Examination Scheduling:** Scheduling exams for students to minimize conflicts and optimize resource allocation, considering factors like room capacity and student preferences.
6. **Constraint-Based Configuration:** Configuring complex products or systems by selecting components and features while satisfying compatibility and constraint rules.
7. **Constraint-Based Diagnosis:** Identifying faults or malfunctions in complex systems by reasoning about observed symptoms and constraints on possible causes.
8. **Robot Motion Planning:** Planning collision-free paths for robots in dynamic environments while considering constraints such as obstacles, robot dynamics, and task objectives.
9. **Circuit Layout Design:** Designing integrated circuits by placing and routing components to optimize performance, power consumption, and area utilization while satisfying layout rules and design constraints.
10. **Network Design and Optimization:** Designing telecommunications or transportation networks to optimize connectivity, bandwidth allocation, and resource utilization while considering constraints such as cost and capacity limits.

Reference Material

- <https://pages.mtu.edu/~nilufer/classes/cs5811/2020-fall/lecture-slides/cs5811-ch06a-csp.pdf>
- <https://www.youtube.com/watch?v=yYQIG9iVvVQ>
- https://www.youtube.com/watch?v=g_T19glKzyU
- <https://www.youtube.com/watch?v=hPeh9-Zf1J8>
- <https://www.youtube.com/watch?v=4Ei20TPtrV4>
- <https://www.youtube.com/watch?v=LuCYmMsLq9k>
- <https://www.youtube.com/watch?v=4UV5abWQFUU>
- <https://www.youtube.com/watch?v=TXJ-k9ljDo0>
- <https://www.youtube.com/watch?v=81z2ANjQcH4&list=PLsOUugYMBBJENfZ3XAToMsg44W7LeUVhF&index=4>
- <https://www.youtube.com/watch?v=DXf6oaknHw&list=PLsOUugYMBBJENfZ3XAToMsg44W7LeUVhF&index=5>